

TaxPulse: AI Tax Lawyer and FBR Filing Agent

Final Year Project Proposal

Session FALL 2026

BS in Artificial Intelligence

Supervisor: Dr Muhammad Rafi



Department of Artificial Intelligence

FAST School of Computing

FAST National University, Karachi Campus

18th September 2026

Project Registration

Project ID (for office use)						
Type of project	☒ Traditional ☐ Industrial ☐ Continuing					
Nature of project	☐ Development ☒ Research & Development ☐ Research					
Sustainable Development Goals(SDGs)	☐ Good Health and Well-Being ☐ Quality Education ☒ Industry, Innovation, and Infrastructure ☐ Gender Equality ☒ Decent Work and Economic Growth ☐ Climate Action					
Area of specialization	☒ Artificial Intelligence (AI) ☐ Data Science and Analytics ☐ Internet of Things (IoT) ☐ Blockchain ☐ Mobile App Development ☒ Web Development ☐ Cybersecurity ☐ Game Development ☒ Natural Language Processing (NLP) ☐ Other					
Project Group Members						
Sr.#	Reg. #	Student Name	CGPA	Email ID	Phone #	Signature
(i)	23K-0061	Abdul Rahman Azam	3.08	K230061@nu.edu.pk	+92 316 3677527	
(ii)	23K-0800	Muhammad Mufeez	3.65	K230800@nu.edu.pk	+92 318 2993401	
(iii)	23K-3033	Uzair Haroon	3.90	K233033@nu.edu.pk	+92 315 2295538	
Declaration: FYP group members have cleared all prerequisite courses For FYP-I as per their degree requirements.						

1. Project Abstract

Filing an income tax return is harder than it looks for an ordinary salaried person in Pakistan. The Income Tax Ordinance, 2001 is amended every year through the Finance Act, the reliefs a person is entitled to are spread across different sections, and the return and wealth statement have to be typed into FBR's IRIS portal by hand, where a single wrong figure can leave the wealth reconciliation unbalanced. TaxPulse is an AI system for resident salaried individuals, and it has two parts. The AI Tax Lawyer is a copilot of four agents that reads the filer's documents, checks which reliefs the law allows them and explains every recommendation with the section it rests on. All tax arithmetic is done by Z3 constraint programs. An LLM helps us write these programs, but a program is accepted only after it matches IRIS's own calculations and passes property checks. The FBR Filing Agent then enters the approved return into IRIS inside the filer's own session, reads every field back and stops before Submit, so the final decision stays with the filer. The project builds on a legal corpus we have already collected: 3,808 files of Pakistani tax law, rules, SROs, rate tables and IRIS guides, with the public part released as an open dataset. We will evaluate

TaxPulse on tax year 2026 returns, measuring agreement with IRIS, accuracy against returns prepared by a tax practitioner, correctness of citations and field-level filing accuracy.

2. Introduction

This project applies artificial intelligence to tax law. Tax is a useful area for AI because the right answer is not a matter of opinion. For a given person and a given tax year, the Income Tax Ordinance, 2001 [1] decides how much tax is due. We take one concrete case of this, the income tax return of a resident salaried individual in Pakistan, and cover the whole path from working out what is owed to entering it into FBR's IRIS portal [2].

The problem matters for two reasons. The first is the size of the tax base. Pakistan counted 241.49 million people in its 2023 census [3], while the copy of FBR's Active Taxpayers List that we downloaded in September 2026 has about 8.9 million entries, and that figure includes companies and associations of persons. The second is fairness. A salaried person who cannot pay a tax consultant has to find the reliefs on their own: the deductible allowance for Zakat (section 60), the tax credits for donations to approved bodies (section 61) and for contributions to an approved pension fund (section 63), and credit for adjustable tax already deducted or collected from them during the year (section 168). Each of these has to be claimed by the filer, with evidence.

At the moment a salaried filer has three options, and none of them solves the whole problem. Online salary tax calculators give a figure for the inputs they receive, but they typically do not explain the law behind the figure, check the wealth statement or point out reliefs the filer has missed. General-purpose chatbots explain tax rules fluently but make mistakes in the arithmetic. On the TaxCalcBench benchmark, state-of-the-art models calculated fewer than a third of the federal returns in a simplified US sample correctly [4], and in a study of tax preparation software, GPT-4o and Claude 3.5 reached worst-case pass rates of only 9 to 15 percent on complex tax-code tasks [5]. The third option is a paid consultant, which brings back exactly the cost the other two were supposed to avoid. In all three cases, filing itself remains manual: someone still has to log in to IRIS and type the figures into the return and the wealth statement by hand.

We think the problem needs two different tools working together. The law is large and it changes every year. Our corpus alone holds 13 dated versions of the Ordinance, the Finance Acts from 2011 to 2026 and 1,029 income-tax SROs, and some provisions that were repealed years ago still appear as current in guides online. Finding the right provision for the right year is a retrieval problem, which suits a language model working over a dated, section-level index of the law. The result, though, is an amount of money that has to be exact, so the arithmetic is left to a solver. Related work exists abroad: neuro-symbolic engines for Indian income tax [6] and for financial compliance cases in Taiwan [7], and Catala, a programming language built for turning statutes such as part of the US income tax code into executable programs [8]. To our knowledge, none of them covers Pakistan's Ordinance or the IRIS portal, and none carries its result through to an actual filing.

3. Problem Statement

Tax calculators for Pakistan already exist, so a missing calculator is not the problem. The problem is that none of the options above gives a salaried filer a correct and fully justified return for a specific tax year and then gets it into IRIS without adding new errors. We see four specific gaps.

Performance gap. Language models read tax law well but calculate with it badly. TaxCalcBench [4] found that models misuse tax tables, make calculation errors and get eligibility wrong. A system whose figures come from a language model cannot give the certainty a tax return needs.

Methodological gap. The usual way to trust generated tax code is to test it against a trusted reference implementation. For Pakistan there is no open, official implementation of the tax computation that a test suite can call. IRIS does calculate tax, but only inside a logged-in session and one return at a time. Gogani-Khiabani et al. [5] faced a similar missing-oracle problem for US tax software and used metamorphic testing [9], where the exact expected output is unknown but the relation between two outputs is known, for example that a higher salary should never produce less tax. We have not found any work that builds a test setup of this kind for Pakistani income tax.

Knowledge gap. Every resident individual who files a return must also file a wealth statement and a wealth reconciliation statement (section 116(2)) [1]. The increase in the filer's net assets over the year has to be explained by the income declared and other inflows, less personal expenses and other outflows. IRIS shows any difference that is left over as the Unreconciled Amount (code 703000) [10], but it does not say where the difference comes from, and an unexplained difference can expose the filer to section 111, which deals with unexplained income or assets. Finding the cause is a search problem, because the question is which entries are missing or wrong. At present that search is left to the filer.

Application gap. Advice alone does not finish the job. Returns are filed through the IRIS web forms and there is no public API for submitting them, so even a correct figure still has to be typed into a portal with many connected screens. That moves the risk of error to the step the filer is least able to check. We have not found a published system that carries a checked computation into IRIS while keeping the filer in control of every change.

The project is therefore built around four research questions.

- RQ1.** Can the provisions that apply to a resident salaried individual for tax year 2026 be written, with an LLM's help, as Z3 constraint programs that agree with IRIS's own calculation on every test case?
- RQ2.** With no public reference calculator, how well do three checks work together as a test oracle: cases observed in IRIS, a second implementation written directly from the statute, and metamorphic properties that Z3 checks for all inputs?
- RQ3.** When a wealth reconciliation does not balance, can a solver identify the missing or mistyped entries that explain the difference, using only figures the filer's documents support?
- RQ4.** Can an approved return be entered into IRIS inside the filer's own session, with every field read back and checked, and with nothing submitted except by the filer?

4. Motivation

The motivation behind TaxPulse is simple: filing correctly costs the most for the people who can least afford a consultant. With this project we want to:

1. cut the manual work, so the filer no longer types dozens of connected figures into IRIS by hand;
2. make the kind of advice that normally needs a paid consultant available to an ordinary salaried person;
3. automate the repetitive part of filing, which is moving between screens, entering fields and checking them, while every decision and the final submission stay with the filer;
4. give the filer a simple chat interface and a review screen that shows every change as a before-and-after comparison before anything is written;
5. make sure every figure comes from a tested constraint program and every recommendation carries its legal citation; and
6. keep the filer's profile, documents, figures and filing history in one place, with each figure linked to the document it came from.

Three things shaped the design.

First, getting the figure right and getting it into IRIS are one job. A correct tax figure that the filer must still copy into dozens of fields by hand only moves the error to the last step, which is why the filing agent is a core part of the project.

Second, TaxPulse helps people claim what the law already gives them, and nothing more. It does not invent deductions or suggest schemes. Section 109 allows the Commissioner to recharacterise a transaction entered into as part of a tax avoidance scheme, and section 111 deals with unexplained income or assets [1]. By working only with reliefs that the filer's own documents support, the system stays clear of both and still gives the filer what they want, which is a lower tax bill that is also correct.

Third, the calendar matters. The last date for filing tax year 2026 returns is 30 September 2026 (section 118). A return can still be filed after that date, but it attracts a penalty under section 182, and under section 100BA a person on the Active Taxpayers List who has not filed by the due date is dealt with under the Tenth Schedule. For us, this means the live TY2026 forms, their validation messages and the figures IRIS computes are best recorded while the season is open. We therefore start that work in the first week and plan to finish it by 30 September.

What we hope to learn is how far formal checking can be taken in an area that is normally handled by informal advice, and how much work it takes to keep such a system correct when the law changes every June.

a. Aims and Objectives

The aim of the project is to build and evaluate an AI system that works out a resident salaried individual's income tax correctly, applies every relief the filer's documents support, explains each figure with the law behind it, and enters the approved return into IRIS with the filer in control. The objectives are:

1. to clean our legal corpus and build a section-level knowledge base of the Income Tax Ordinance, 2001 for tax years 2026 and 2027, with every record tagged by section and tax year;
2. to build a Z3 tax engine for tax year 2026 that covers the scope in Section 4(b), with a planning mode for tax year 2027, and to accept it only when it agrees with IRIS on every captured test case;
3. to build a wealth reconciliation check that points to the likely causes of an unreconciled amount without inventing any figure;
4. to build the four-agent AI Tax Lawyer copilot, which extracts the filer's figures from documents and chat, recommends reliefs with citations and never changes a number produced by the engine;
5. to build the FBR Filing Agent, which enters the approved return into IRIS inside the filer's own session, reads every field back and stops before Submit; and
6. to evaluate the system against IRIS, against returns prepared by a tax practitioner and against LLM-only baselines.

b. Scope of the Project

TaxPulse is meant for resident individuals whose main income is salary. For tax year 2026 (1 July 2025 to 30 June 2026) it will handle:

1. income from salary (section 12);
2. profit on bank deposits and savings (section 7B);
3. dividends (section 5);
4. capital gains on listed shares and mutual fund units (sections 37A and 100B);
5. rental income (section 15); and
6. the wealth statement and wealth reconciliation (section 116), together with credit for adjustable tax deducted or collected during the year (section 168).

For tax year 2027, the current year, TaxPulse will also work in a planning mode that shows how much tax the filer can still save before 30 June 2027 through steps the law allows, such as contributions to an approved pension fund (section 63) or donations to approved bodies (section 61).

Business income, foreign income and assets (section 116A), agricultural income, capital gains on property, non-resident individuals, companies, associations of persons, sales tax and provincial taxes are outside the scope of this project. TaxPulse prepares and enters the return, but it never submits it. Submission, and any tax payment, remain with the filer.

5. Proposed Methodology

The method follows the architecture in Section 7 and runs on two tracks. The build track produces, once for each tax year, a tax engine that has passed its tests. The per-filer track uses that engine for each return. Two rules decide the order of work. Nothing downstream is trusted until the engine passes its tests, so every later phase waits on that gate. And the live IRIS capture has to happen while the TY2026 season is open, so it runs alongside the first phase instead of after it.

Data and datasets. We did not start from an off-the-shelf dataset. We built the corpus ourselves by crawling FBR’s website and other official sources, and Table 1 shows what exists already, what is still to be captured and what each part is used for.

Table 1. Project datasets, their size and their role in the system.

Dataset	Content and size	Role in the system
Legal source corpus	3,808 files (7.2 GB) in 15 tiers: 13 dated consolidations of the Ordinance, the Income Tax Rules 2002, Finance Acts 2011-2026, 1,029 income-tax SROs (1984-2026), circulars, return forms, IRIS guides, 133 case-law files and 72 tax treaties. This is 85% of the 3,798 income-tax documents we found on FBR’s website. 3,446 of the files are indexed with their source URL and SHA-256 checksum.	The legal ground truth, and the source of every citation.
Section-level Ordinance records	771 JSONL records from the 30 June 2026 consolidation: 491 section records (380 with operative text; the rest are stubs for omitted or renumbered sections, plus 38 table-of-contents entries that were parsed as sections and will be removed), 80 schedule blocks and 200 Second Schedule clauses. Each record keeps its section, heading, chapter, page and the Finance Act footnotes printed on that page.	The unit for embedding and retrieval. The 30 June 2026 text is tax year 2027 law, so for tax year 2026 we add records from the 20 February 2026 consolidation, which reflects the Finance Act 2025 [11], together with the Finance Act 2026 provisions that apply from tax year 2026, such as section 7G [12].
Income Tax Rules 2002 records	219 JSONL records from the 24 November 2023 consolidation.	Subordinate legislation, read together with the SROs issued after 2023 [13].
Withholding rate table	303 rows for tax years 2026 (149 rows) and 2027 (154 rows), with the ATL rate, the non-ATL rate, the late-filer rate where one exists, and the First Schedule reference for each rate.	The rate input to the tax engine, used instead of rate tables copied out of the Ordinance PDF.
Active Taxpayers List	FBR’s list of active taxpayers, published each March and updated every Sunday (Rule 81B). Our September 2026 copy has about 8.9 million entries. It is kept private and used only to look up filer status.	Filer status decides which withholding rates apply (section 100BA and the Tenth Schedule).
IRIS field maps (to be captured)	Field codes, control types, links between screens, validation messages and IRIS’s computed figures for the TY2026 return under section 114(1) and the wealth statement under section 116. SRO 1495(I)/2026 [10] already gives the official screen layouts and field codes; live capture adds how the screens behave.	The schema the filing agent writes to, and the main test oracle for the engine.
Synthetic taxpayer profiles (to be built)	Generated profiles that cross the slab boundaries, with each relief present and absent, more than one employer, the in-scope non-salary income, and wealth statements with deliberately planted errors.	The evaluation set. No real taxpayer’s data is used.

We treat provenance as part of the data. Most files are indexed with the URL they came from and a SHA-256 checksum, the 39 dead links on FBR’s own server are listed rather than dropped, and an integrity pass opens every document. That pass found 153 old SROs that FBR serves as HTML behind PDF links and 119 PDFs with extra bytes before the PDF header, and both problems have been repaired. A citation is only as good as the document behind it.

Phases. The work is split into six phases and two closing activities. Each phase ends with a deliverable and an exit test that someone outside the team can check (Table 2), and Table 6 places them on the calendar.

Table 2. Phases, deliverables and exit tests.

Phase	Weeks	Deliverable and exit test
Phase 1	W1-W2	Knowledge base and scope lock. Clean the Ordinance records (remove the misparsed entries and split the footnotes per section), add tax year 2026 law, and build section-level retrieval with tax-year tags. Exit test: a query returns the correct section for the correct tax year.
Phase 2	W1-W6	Live IRIS capture. Record the field maps of the TY2026 return and wealth statement from an authenticated IRIS session, and IRIS’s computed figures for known inputs. Test entries are never submitted. Exit test: a recorded case reproduces IRIS’s computed tax from the captured data alone.
Phase 3	W3-W8	Tax engine for TY2026. An LLM drafts Python and Z3 programs from the retrieved sections, and each draft is checked against the IRIS cases, the statute-based checker and the metamorphic properties, then patched until it passes. Exit test: 100% agreement on the captured cases, and every property proved or its counterexample explained.
Phase 4	W7-W11	Reconciliation check, relief recommender and planning mode. Encode the section 116 reconciliation, find the likely causes of an unreconciled amount, recommend a relief only when a citation supports it, and add TY2027 planning with Z3 Optimize. Exit test: in synthetic profiles with planted errors, the planted error is among the top three causes suggested.
Phase 5	W10-W13	Copilot agents and document intake. Assemble the four agents with their numeric guards, and read salary certificates, bank statements and the Summary of Economic Transactions into the case file. Exit test: from a chat and uploaded documents, the copilot produces a plan in which every figure traces back to the engine and every recommendation has a citation.
Phase 6	W11-W16	FBR Filing Agent and integration. Choose between a Chrome extension and Playwright, then build navigation, filling with read-back checks, the review screen and undo. Exit test: a full return is filled up to the point of submission, with every field checked.
Evaluation	W14-W16	All outcomes in Section 11 measured against the baselines, including the practitioner comparison and the ablations.
Write-up	W15-W18	Thesis, demonstration and viva.

Oracle strategy. Testing needs a trusted reference, so we use three of them, in order of authority. The first is IRIS itself. A known profile is entered into a TY2026 return draft, IRIS’s Calculate button is pressed, and the computed figures are recorded as a regression case; these drafts are never submitted. The second is a separate implementation that a different team member writes by hand from the Ordinance and the Finance Act, without the LLM, so that a misreading in one is unlikely to be repeated in the other. The third is a set of metamorphic properties [9], such as “tax never falls when salary rises” and “splitting the same salary between two employers does not change the tax due”. Z3 checks these for every input the engine allows, which is stronger than testing a sample of cases. Questions of

judgement that none of these can settle, for example whether a particular donation qualifies, go to the tax practitioner working with us.

The engine is accepted only when it agrees with IRIS on 100% of the captured cases. If IRIS and our reading of the statute ever disagree, for example over a rounding rule, we record the case, document the cause and show it to the filer. A tax figure is either right or wrong, so we do not set a threshold below 100%.

6. Functional and Non-Functional Requirements

Table 3 lists what the system must do, and Table 4 lists the qualities it must keep while doing it. Some of the non-functional requirements are really design rules: a system that met every functional requirement but broke NFR1, NFR3 or NFR5 could not be used.

Table 3. Functional requirements.

ID	Requirement
FR1	Search the legal knowledge base and return the relevant sections, each with its instrument, section number and tax year.
FR2	Extract figures from the chat and from uploaded documents (salary certificates, bank statements, IRIS's Summary of Economic Transactions and payment receipts), convert lakh and crore to rupees, and link each figure to the page or row it came from.
FR3	Ask the filer to confirm the extracted figures and to explain any bank credit that cannot be classified.
FR4	Turn the filer's case file into typed Z3 constraints for the selected tax year.
FR5	Compute the tax, the credits and the refund or amount payable using the tested engine for that tax year.
FR6	List the reliefs the filer's facts support (deductible allowances, tax credits and credit for adjustable tax), each with its citation.
FR7	In planning mode for tax year 2027, show how much tax the filer can still save through approved pension fund contributions and donations within the legal limits, using Z3 Optimize.
FR8	Detect an unreconciled amount in the wealth reconciliation and list its most likely causes, using only entries that the filer's documents support.
FR9	Look up the filer's status on the latest Active Taxpayers List and apply the matching withholding rates.
FR10	Produce a plan with a before-and-after comparison, the sections relied on and a plain-language explanation, and let the filer accept or reject each recommendation.
FR11	Open the TY2026 return and wealth statement in the filer's own IRIS session and move between their screens.
FR12	Fill each field, read the value back from the page, and compare IRIS's Calculate result with the engine; any mismatch stops the process.
FR13	Show every intended field change as a preview that the filer can deselect, and undo any change after it is applied.
FR14	Export a fill sheet (field code and value) as a fallback when direct filling is not possible.
FR15	Stop before Submit. Submission and any tax payment are done by the filer.

Table 4. Non-functional requirements.

ID	Requirement
NFR1	Correctness. Every figure shown to the filer comes from the tax engine, never from a language model. The model writes its explanation around placeholders, the figures are filled in by code, and any number in the text that does not match an engine value blocks the plan.
NFR2	Traceability. Every recommendation names the instrument, section and tax year it relies on. If retrieval finds no supporting provision, the recommendation is not made.
NFR3	Privacy. Customer documents are encrypted at rest, CNIC, NTN and account numbers are masked in logs, and only synthetic data is used for testing and evaluation.
NFR4	Security. The system never asks for, stores or sends IRIS passwords. The filer logs in and completes any verification in their own browser, and session cookies are not kept after the session ends.
NFR5	Safety. Controls that submit, verify or pay, that delete a saved return, or that log out are on a blocked list that the automation cannot override, and the agent acts only while the filer is present.
NFR6	Maintainability. Rules and field maps are versioned per tax year, so a new Finance Act means adding and testing a new rule set while the rest of the system stays the same.
NFR7	Robustness. A write that cannot be confirmed by reading it back is reported as unverified. Navigation follows recorded routes, so a change in IRIS’s layout is fixed by updating data.
NFR8	Usability and accessibility. The review screen supports keyboard use and screen readers (ARIA live regions), visible focus, WCAG AA contrast and reduced motion, and it detects the current view again when IRIS changes page.
NFR9	Reproducibility. Every result stores the engine version, the corpus snapshot and the inputs behind it, so a filed return can be re-derived later.
NFR10	Auditability. Every approval the filer gives is logged with a timestamp.

7. System Architecture

TaxPulse has two parts, matching the two halves of its title. The AI Tax Lawyer is a copilot of four agents that works out the return and explains it, and the FBR Filing Agent enters the approved return into IRIS. Both depend on a knowledge base of the law and on a tax engine that is built and tested once for each tax year, before any filer uses it. The main components are:

1. **Knowledge base.** The Ordinance [1], the Finance Acts [11], [12], the Rules, SROs and circulars, split into section-level records with tax-year tags and indexed for retrieval [14].
2. **Tax engine, built once per tax year.** An LLM drafts Python and Z3 [15] programs from the retrieved sections. The drafts are checked against the IRIS cases, the statute-based checker and the metamorphic properties, patched until they pass, and then frozen as the engine for that tax year. No LLM is involved when the engine runs for a filer.
3. **AI Tax Lawyer (four-agent copilot).** CallerAgent reads the chat and the documents and builds the case file. ConstraintAgent turns the case file into typed Z3 constraints for the selected tax year. ExecuteAgent runs the engine, including Z3 Optimize [16] in planning mode and the reconciliation check. ReasoningAgent explains the result with citations, under the numeric guard described in NFR1. The agents follow the reason-and-act pattern of ReAct [17], but acting always means calling a tool, never doing arithmetic in text.
4. **FBR Filing Agent.** It moves through the filer’s own IRIS session, fills and reads back each field, and compares IRIS’s Calculate result with the engine. It will be built

- either as a Chrome extension (Manifest V3) [18] or with Playwright [19]; we will choose between them in Phase 6, after trying both on the captured IRIS forms.
5. **Human checkpoints.** The filer confirms the extracted facts, accepts or rejects each recommendation, and finally reviews the full field-level comparison in IRIS and presses Submit. The last checkpoint is part of the design and cannot be switched off.

Fig. 1 shows the build track at the top and the per-filer track below it, together with the three checkpoints.

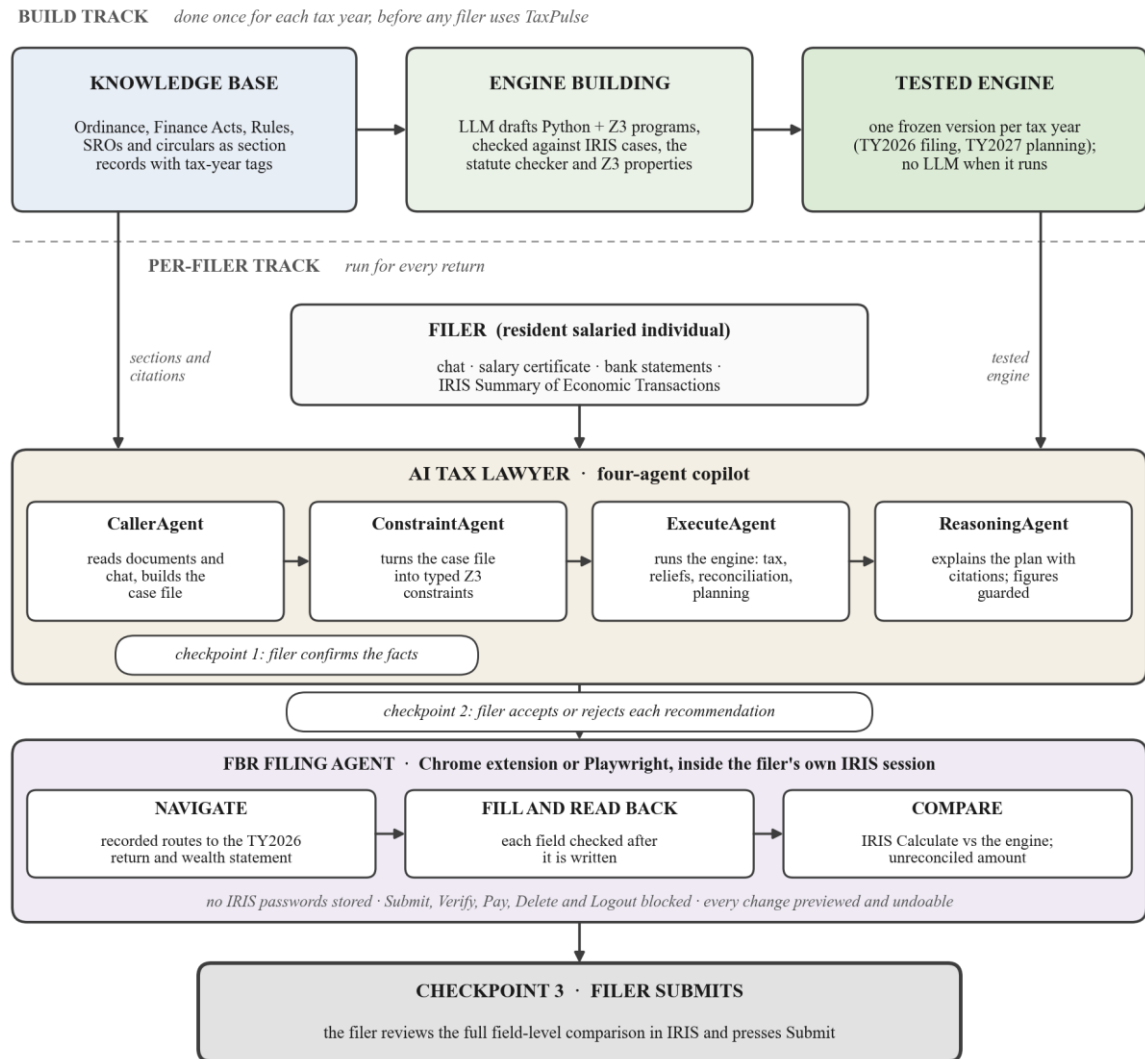


Fig. 1. TaxPulse architecture: a build track that produces a tested engine for each tax year, and a per-filer track with three human checkpoints.

How IRIS forms affect the filing agent. IRIS 2.0 is built with Angular and Angular Material; its page source loads the Angular runtime bundles and uses Angular Material's typography class. In Angular's reactive forms, the FormControl model is the source of truth for each field [20], and it is updated from events raised by the input element. If a script simply sets an element's value, no event is raised and the model is not updated, so the field can be saved blank. The filing agent therefore enters values through real input events. Playwright's fill method does this by design, since it triggers an input event after filling [19], and an extension can do the same by dispatching input and change events. Where a field is an Angular Material select, which has no plain input to write to, the agent opens it

and clicks the option, as a person would. Any value that cannot be confirmed by reading it back is reported as unverified.

The knowledge base holds only the law. Taxpayer data stays in the filer’s case file and is protected as described in NFR3 and NFR4, and IRIS is only ever used inside a session that the filer opened.

8. Individual Tasks

Work is divided by workstream (Table 5). Each member owns the activities listed against their name, and all three share the evaluation, the reports and the final demonstration.

Table 5. Individual tasks and tentative dates.

Team Member	Activity	Tentative Date
Abdul Rahman Azam 23K-0061 <i>Law and knowledge base</i>	Clean the Ordinance records, add tax year 2026 law and build retrieval with tax-year tags	20 Aug to 2 Sep 2026
	Write the statute-based checker for tax year 2026 by hand, without the LLM	3 Sep to 14 Oct 2026
	Citation checks for the relief recommender, and the tax year 2027 rule set for planning mode	1 Oct to 4 Nov 2026
	Retrieval and citation support for the ReasoningAgent	22 Oct to 18 Nov 2026
	Evaluation: citation soundness and the practitioner comparison	19 Nov to 9 Dec 2026
Muhammad Mufeez 23K-0800 <i>Tax engine and agents</i>	Z3 tax engine for tax year 2026 and its metamorphic properties	3 Sep to 14 Oct 2026
	Wealth reconciliation check, and planning mode with Z3 Optimize	1 Oct to 4 Nov 2026
	Four-agent copilot with numeric guards	22 Oct to 18 Nov 2026
	Evaluation: agreement with IRIS, reconciliation diagnosis, baselines and ablations	19 Nov to 9 Dec 2026
Uzair Haroon 23K-3033 <i>Intake, filing and app</i>	Live IRIS capture of the TY2026 return and wealth statement	20 Aug to 30 Sep 2026
	Backend API (FastAPI or Node.js) and case-file storage	1 Oct to 4 Nov 2026
	Document intake: salary certificates, bank statements and the Summary of Economic Transactions	22 Oct to 18 Nov 2026
	FBR Filing Agent: navigation, fill and read-back, review screen and undo	29 Oct to 9 Dec 2026
	Evaluation: field-level filing accuracy	19 Nov to 9 Dec 2026
All three members	Thesis, demonstration and viva	26 Nov to 23 Dec 2026

9. Gantt Chart

The schedule runs for eighteen weeks, from 20 August to 23 December 2026, and Table 6 places the phases of Table 2 on that calendar. The first six weeks carry two streams of work at once. This is deliberate: the live IRIS capture has to happen while the TY2026 filing season is open, so it cannot wait for the knowledge base to be finished. Phases 3 to 6 then overlap, and each one starts as soon as the phase before it has produced what it needs.

Table 6. Gantt chart of project phases, weeks 1 to 18 (20 August to 23 December 2026).

2026	Aug		Sep				Oct				Nov					Dec		
Phase	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Phase 1 Knowledge base and scope lock																		
Phase 2 Live IRIS capture																		
Phase 3 Tax engine for TY2026																		
Phase 4 Reconciliation, recommender and planning																		
Phase 5 Copilot agents and document intake																		
Phase 6 FBR Filing Agent and integration																		
Evaluation																		
Thesis, demonstration and viva																		

The critical path runs through Phase 2. The last date for TY2026 returns, 30 September 2026, falls in week 6, and later phases depend on the field maps and IRIS cases recorded before then, so Phase 2 starts in week 1 and is treated as fixed. If anything is missed, SRO 1495(I)/2026 [10] still provides the official screen layouts and field codes, and the statute-based checker and the metamorphic properties can carry the testing until more IRIS cases are recorded. Evaluation starts once the filing agent can fill a complete return, and it runs alongside the final integration work.

10. Tools and Technologies

The tools follow the same split as the architecture: a Python side for the law, the engine and the agents, and a web side that has to work inside the browser. Table 7 lists what each part uses and why.

Table 7. Tools and technologies.

Part	Tools and technologies	Purpose
Tax engine	Python; Microsoft Z3 (z3-solver) with Z3 Optimize	Exact tax arithmetic, property checks and planning-mode optimization.
Retrieval	ChromaDB or a similar vector database; sentence-transformer embeddings	Section-level storage and search of the law, with tax-year tags.
Agent runtime	Python agent framework with tool calling; a provider-independent LLM interface	The four copilot agents, without tying the system to one LLM vendor.
Document intake	PDF and image text extraction, with OCR for scanned pages	Reading salary certificates, bank statements and payment receipts into the case file, with page references.
Backend API	FastAPI (Python) or Node.js	Case-file storage, agent orchestration and data for the review screen.
Filing agent	Chrome extension (Manifest V3, JavaScript) or Playwright	Navigation, filling and read-back inside the filer's own IRIS session.
Filing-agent testing	An offline copy of the IRIS return screens, run in a real browser	Regression tests without touching the live portal.

Part	Tools and technologies	Purpose
Data engineering	Python crawlers and parsers (PyMuPDF, openpyxl), manifest-driven downloaders, integrity and repair scripts	Building and refreshing the corpus from official sources.
Data distribution	Hugging Face Datasets; Git LFS	A public mirror of the corpus, and private storage for the taxpayer lists that are kept out of it.
Engineering	Git and GitHub; automated test suites; regression runs against the captured IRIS cases	Version control, testing and reproducible builds.

11. Expected Deliverables

The project delivers a set of working artefacts and a set of measured results.

Expected outcomes. We compare TaxPulse with three baselines: an LLM answering from its prompt alone, an LLM with standard retrieval that does its own arithmetic, and online salary tax calculators (for the tax figure only). All tests use synthetic profiles that span the salaried cases in scope. We expect to report:

1. Agreement with IRIS: 100% on the captured cases, with every disagreement explained.
2. Property checks: every stated property either proved by Z3 or its counterexample explained.
3. Accuracy against the practitioner: the share of profiles where TaxPulse reaches the same reliefs and the same final tax as the practitioner's independently prepared return, with each disagreement classed as a system error, a practitioner judgement or a real ambiguity in the law.
4. Filing accuracy: fields written correctly, written wrongly and skipped, compared with the reference return, both on the offline copy of IRIS and in live draft sessions that are never submitted.
5. Reconciliation diagnosis: the share of synthetic wealth statements with a planted error where the planted error is among the top three causes TaxPulse suggests.
6. Citation soundness: the share of recommendations whose citation is in force for the tax year and actually supports the recommendation, as judged by the practitioner.
7. Maintenance cost: the number of draft-check-patch rounds needed to reach full agreement, as an estimate of the work each new Finance Act will need, along with ablations that remove retrieval, the solver and the numeric guard one at a time.

Deliverable artefacts.

1. A section-level knowledge base of the Income Tax Ordinance, 2001 for tax years 2026 and 2027, with citations and provenance.
2. The legal corpus and its machine-readable layer, published on Hugging Face with source URLs and checksums; the public part is already online.
3. Versioned field maps of the TY2026 return and wealth statement, recorded from IRIS during the filing season.
4. A tested Python and Z3 tax engine for resident salaried individuals for tax year 2026, with its test suite and documented differences.

5. A planning mode for tax year 2027.
6. A wealth reconciliation check that suggests the likely causes of an unreconciled amount.
7. A relief recommender that recommends only what a citation supports.
8. The four-agent AI Tax Lawyer copilot, which produces plans with before-and-after comparisons and legal references.
9. The FBR Filing Agent, with preview, read-back checks, undo and the list of blocked controls.
10. An evaluation report covering all the outcomes above.
11. Technical documentation, the test suites, and a final demonstration that goes from a chat with the filer to a return that is ready for the filer to submit.

12. References

- [1] Government of Pakistan, *Income Tax Ordinance, 2001 (XLIX of 2001)*, consolidated versions as amended up to 20 Feb. 2026 and 30 Jun. 2026. Islamabad, Pakistan: Federal Board of Revenue, 2026.
- [2] Federal Board of Revenue, “IRIS 2.0.” [Online]. Available: <https://iris.fbr.gov.pk> (accessed Sep. 18, 2026).
- [3] Pakistan Bureau of Statistics, “Announcement of results of 7th Population and Housing Census-2023 ‘The Digital Census’,” Islamabad, Pakistan, Aug. 5, 2023. [Online]. Available: <https://www.pbs.gov.pk/wp-content/uploads/2024/02/Press-Release.pdf>
- [4] M. R. Bock, K. Molisee, Z. Ozer, and S. Shah, “TaxCalcBench: Evaluating frontier models on the tax calculation task,” arXiv:2507.16126, Jul. 2025.
- [5] S. Gogani-Khiabani, A. Trivedi, D. Saha, and S. Tizpaz-Niari, “An LLM agentic approach for legal-critical software: A case study for tax prep software,” in *Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE)*, Rio de Janeiro, Brazil, Apr. 2026. [Online]. Available: arXiv:2509.13471
- [6] K. Veeramani, Allen Joseph N, and Pavithran M, “Neuro-symbolic reasoning engine for tax optimisation,” *Frontiers in Artificial Intelligence*, vol. 9, Art. no. 1802755, May 2026, doi: 10.3389/frai.2026.1802755.
- [7] Y.-S. Hsia, F. Yu, and J.-H. R. Jiang, “Neuro-symbolic compliance: Integrating LLMs and SMT solvers for automated financial legal analysis,” in *Proc. 2nd ACM Int. Conf. AI-Powered Software (AIware)*, Seoul, South Korea, Nov. 2025. [Online]. Available: arXiv:2601.06181
- [8] D. Merigoux, N. Chataing, and J. Protzenko, “Catala: A programming language for the law,” *Proc. ACM on Programming Languages*, vol. 5, no. ICFP, Art. no. 77, pp. 1-29, Aug. 2021, doi: 10.1145/3473582.
- [9] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, “Metamorphic testing: A review of challenges and opportunities,” *ACM Computing Surveys*, vol. 51, no. 1, Art. no. 4, pp. 4:1-4:27, 2018.
- [10] Federal Board of Revenue, *Notification S.R.O. 1495(I)/2026 (Income Tax)*, adding the electronic return forms for tax year 2026 to the Second Schedule of the Income Tax Rules, 2002. Islamabad, Pakistan, Sep. 2, 2026.
- [11] Government of Pakistan, *The Finance Act, 2025*. Islamabad, Pakistan, 2025.
- [12] Government of Pakistan, *The Finance Act, 2026*. Islamabad, Pakistan, 2026.

- [13] Government of Pakistan, *Income Tax Rules, 2002*, consolidated version as amended up to 24 Nov. 2023. Islamabad, Pakistan: Federal Board of Revenue, 2023.
- [14] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459-9474.
- [15] L. de Moura and N. Bjørner, “Z3: An efficient SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS, vol. 4963. Berlin, Germany: Springer, 2008, pp. 337-340.
- [16] N. Bjørner, A.-D. Phan, and L. Fleckenstein, “vZ - An optimizing SMT solver,” in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, LNCS, vol. 9035. Berlin, Germany: Springer, 2015, pp. 194-199.
- [17] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.
- [18] Google, “Extensions / Manifest V3,” Chrome for Developers. [Online]. Available: <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3> (accessed Sep. 18, 2026).
- [19] Microsoft, “Locator,” Playwright documentation. [Online]. Available: <https://playwright.dev/docs/api/class-locator> (accessed Sep. 18, 2026).
- [20] Google, “Reactive forms,” Angular documentation. [Online]. Available: <https://angular.dev/guide/forms/reactive-forms> (accessed Sep. 18, 2026).